CSA 1737 – ARTIFICIAL INTELLIGENCE

Laboratory Record – Experiments 1 to 5

Ex. No. 1: Simple Calculator

AIM

To write a Python program to implement a simple calculator that performs addition, subtraction, multiplication and division.

ALGORITHM

Step 1: Start the program.

Step 2: Define four functions: add(), subtract(), multiply() and divide().

Step 3: In divide(), check whether the second number is zero; if so, return an error message.

Step 4: Display the menu of operations (1. Add, 2. Subtract, 3. Multiply, 4. Divide).

Step 5: Read the user's choice and the two numbers.

Step 6: Call the function matching the choice and display the result.

Step 7: If the choice is not between 1 and 4, display "Invalid input".

Step 8: Stop the program.

CODE

```
# Experiment 1: Simple Calculator
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Error: Division by zero is not allowed"
    return a / b

print("===== SIMPLE CALCULATOR =====")
print("Select operation:")
print("1. Add")
print("2. Subtract")
print("3. Multiply")
print("4. Divide")

choice = input("Enter choice (1/2/3/4): ")
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

if choice == '1':
    print(num1, "+", num2, "=", add(num1, num2))
elif choice == '2':
    print(num1, "-", num2, "=", subtract(num1, num2))
```

```
elif choice == '3':
    print(num1, "*", num2, "=", multiply(num1, num2))
elif choice == '4':
    print(num1, "/", num2, "=", divide(num1, num2))
else:
    print("Invalid input")
```

OUTPUT

```
===== SIMPLE CALCULATOR =====
Select operation:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice (1/2/3/4): 1
Enter first number: 25
Enter second number: 15
25.0 + 15.0 = 40.0


===== SIMPLE CALCULATOR =====
Select operation:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice (1/2/3/4): 4
Enter first number: 20
Enter second number: 0
20.0 / 0.0 = Error: Division by zero is not allowed
```

RESULT

Thus the Python program for a simple calculator was written and executed successfully, and the output was verified.

Ex. No. 2: Addition of Two Matrices

AIM

To write a Python program to add two matrices.

ALGORITHM

Step 1: Start the program.

Step 2: Read the number of rows and columns.

Step 3: Read the elements of Matrix A and Matrix B using nested loops.

Step 4: Create a result matrix of the same size, initialised with zeros.

Step 5: For every position (i, j), compute result[i][j] = A[i][j] + B[i][j].

Step 6: Display Matrix A, Matrix B and the resultant matrix.

Step 7: Stop the program.

CODE

```
# Experiment 2: Add Two Matrices
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

print("Enter elements of Matrix A:")
A = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"A[{i}][{j}]: ")))
    A.append(row)

print("Enter elements of Matrix B:")
B = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"B[{i}][{j}]: ")))
    B.append(row)

# Result matrix initialised with zeros
result = [[0 for _ in range(cols)] for _ in range(rows)]

for i in range(rows):
    for j in range(cols):
        result[i][j] = A[i][j] + B[i][j]

print("\nMatrix A:")
for r in A:
    print(r)
print("\nMatrix B:")
for r in B:
    print(r)
print("\nSum of Matrix A and Matrix B:")
for r in result:
    print(r)
```

OUTPUT

```
Enter number of rows: 2
Enter number of columns: 3
Enter elements of Matrix A:
A[0][0]: 1
A[0][1]: 2
A[0][2]: 3
A[1][0]: 4
A[1][1]: 5
A[1][2]: 6
Enter elements of Matrix B:
B[0][0]: 9
B[0][1]: 8
B[0][2]: 7
B[1][0]: 6
B[1][1]: 5
B[1][2]: 4

Matrix A:
[1, 2, 3]
[4, 5, 6]

Matrix B:
[9, 8, 7]
[6, 5, 4]

Sum of Matrix A and Matrix B:
[10, 10, 10]
[10, 10, 10]
```

RESULT

Thus the Python program to add two matrices was written and executed successfully, and the output was verified.

Ex. No. 3: Transpose of a Matrix

AIM

To write a Python program to find the transpose of a given matrix.

ALGORITHM

Step 1: Start the program.

Step 2: Read the number of rows and columns of the matrix.

Step 3: Read the matrix elements using nested loops.

Step 4: Create a transpose matrix having columns number of rows and rows number of columns.

Step 5: For every element, assign transpose[j][i] = matrix[i][j].

Step 6: Display the original matrix and the transposed matrix.

Step 7: Stop the program.

CODE

```
# Experiment 3: Transpose of a Matrix
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

print("Enter elements of the matrix:")
matrix = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(input(f"Element [{i}][{j}]: ")))
    matrix.append(row)

# Transpose matrix has 'cols' rows and 'rows' columns
transpose = [[0 for _ in range(rows)] for _ in range(cols)]

for i in range(rows):
    for j in range(cols):
        transpose[j][i] = matrix[i][j]

print("\nOriginal Matrix:")
for r in matrix:
    print(r)
print("\nTranspose of the Matrix:")
for r in transpose:
    print(r)
```

OUTPUT

```
Enter number of rows: 2
Enter number of columns: 3
Enter elements of the matrix:
Element [0][0]: 1
Element [0][1]: 2
Element [0][2]: 3
Element [1][0]: 4
Element [1][1]: 5
Element [1][2]: 6
```

```
Original Matrix:
[1, 2, 3]
[4, 5, 6]

Transpose of the Matrix:
[1, 4]
[2, 5]
[3, 6]
```

RESULT

Thus the Python program to find the transpose of a matrix was written and executed successfully, and the output was verified.

Ex. No. 4: Sort the Words of a Sentence in Alphabetical Order

AIM

To write a Python program to sort the words of a given sentence in alphabetical order.

ALGORITHM

Step 1: Start the program.

Step 2: Read a sentence from the user.

Step 3: Split the sentence into a list of words using split().

Step 4: Sort the list using sort() with key=str.lower so that upper and lower case words are treated alike.

Step 5: Display each sorted word and the sorted sentence using join().

Step 6: Stop the program.

CODE

```
# Experiment 4: Sort the words of a sentence in alphabetical order
sentence = input("Enter a sentence: ")

# Split into words and convert to lowercase for case-insensitive sorting
words = sentence.split()
words.sort(key=str.lower)

print("\nThe sorted words are:")
for word in words:
    print(word)

print("\nSorted sentence:", " ".join(words))
```

OUTPUT

```
Enter a sentence: the quick brown fox jumps over a lazy dog

The sorted words are:
a
brown
dog
fox
jumps
lazy
over
quick
the

Sorted sentence: a brown dog fox jumps lazy over quick the
```

RESULT

Thus the Python program to sort the words of a sentence in alphabetical order was written and executed successfully, and the output was verified.

Ex. No. 5: List Operations

AIM

To write a Python program to implement list operations: nested list, length, concatenation, membership, iteration, indexing and slicing.

ALGORITHM

Step 1: Start the program.

Step 2: Create a nested list and access inner elements using multiple indexes.

Step 3: Find the number of elements in a list using len().

Step 4: Join two lists using the + operator and repeat a list using the * operator.

Step 5: Check whether an item is present in a list using the in and not in operators.

Step 6: Traverse the list using a for loop (with and without enumerate()).

Step 7: Access elements using positive and negative indexes.

Step 8: Extract parts of the list using slicing [start:stop:step].

Step 9: Stop the program.

CODE

```
# Experiment 5: List Operations
# 1. Nested list
nested = [1, 2, [3, 4, [5, 6]], 7]
print("Nested List        :", nested)
print("Nested element [2][1]    :", nested[2][1])
print("Deep nested element [2][2][0] :", nested[2][2][0])

# 2. Length
fruits = ["apple", "banana", "cherry", "mango"]
print("\nList               :", fruits)
print("Length             :", len(fruits))

# 3. Concatenation
list1 = [10, 20, 30]
list2 = [40, 50]
print("\nConcatenation      :", list1 + list2)
print("Repetition         :", list1 * 2)

# 4. Membership
print("\n'banana' in fruits     :", "banana" in fruits)
print("'grape' in fruits      :", "grape" in fruits)
print("'grape' not in fruits  :", "grape" not in fruits)

# 5. Iteration
print("\nIteration:")
for item in fruits:
    print(item)
print("Iteration with index:")
for i, item in enumerate(fruits):
    print(i, "->", item)

# 6. Indexing
nums = [10, 20, 30, 40, 50, 60]
```

```
print("\nList               :", nums)
print("First element nums[0]  :", nums[0])
print("Last element nums[-1]  :", nums[-1])

# 7. Slicing
print("\nSlicing:")
print("nums[1:4]  :", nums[1:4])
print("nums[:3]   :", nums[:3])
print("nums[3:]   :", nums[3:])
print("nums[::2]  :", nums[::2])
print("nums[::-1] :", nums[::-1])
```

OUTPUT

```
Nested List        : [1, 2, [3, 4, [5, 6]], 7]
Nested element [2][1]    : 4
Deep nested element [2][2][0] : 5

List               : ['apple', 'banana', 'cherry', 'mango']
Length             : 4

Concatenation      : [10, 20, 30, 40, 50]
Repetition         : [10, 20, 30, 10, 20, 30]

'banana' in fruits     : True
'grape' in fruits      : False
'grape' not in fruits  : True

Iteration:
apple
banana
cherry
mango
Iteration with index:
0 -> apple
1 -> banana
2 -> cherry
3 -> mango

List               : [10, 20, 30, 40, 50, 60]
First element nums[0]  : 10
Last element nums[-1]  : 60

Slicing:
nums[1:4]  : [20, 30, 40]
nums[:3]   : [10, 20, 30]
nums[3:]   : [40, 50, 60]
nums[::2]  : [10, 30, 50]
nums[::-1] : [60, 50, 40, 30, 20, 10]
```

RESULT

Thus the Python program to implement list operations was written and executed successfully, and the output was verified.